Data In, Predictions Out
How FireWatch's Data Became a Model, and How the Model Becomes an Alert

The third and final bridge: exactly what data was fed to the model, how every raw image became a labeled training example, precisely what happens to a live camera frame before it reaches the network, how MobileNetV3-Small's raw output becomes a probability, and the exact decision rules FireWatch layers on top before ever sounding an alarm - grounded entirely in the project's own training report and engineering logs.

Prerequisites: "Convolutional Neural Networks - From First Principles to Full Depth" and "MobileNetV3-Small - Bridging Plain CNNs to FireWatch's Production Model." This document assumes both.

Table of Contents

0. What This Document Adds
1. Where the Data Actually Came From
2. From Raw Images to Labeled Training Examples
3. Data Augmentation - Manufacturing Variety From Fixed Photos
4. What Happens to a Live Camera Frame Before It Reaches the Model
5. From Raw Logits to a Decision - Softmax, Thresholds, and the Argmax-AND Rule
6. Temporal Voting - Turning One Noisy Frame Into a Stable Verdict
7. The Complete End-to-End Trace, One Frame at a Time
8. Why These Specific Rules Exist - the False-Positive Investigations Behind Them
9. What Recall and Precision Actually Mean - the Numbers Behind Every Metric
10. Quick Reference - Every Number, in One Place

0. What This Document Adds

The first document explained how a CNN computes at all - convolution, activations, pooling, backpropagation. The second explained exactly which CNN FireWatch uses, and why - MobileNetV3-Small's specific architecture, and the transfer-learning setup that adapted it to this task. Neither document answered three questions that sit before and after the model itself: what data actually taught this model anything, what precise numerical steps turn a live camera frame into something the model can read, and what precise, non-model logic turns the model's raw output into an actual alert. This document answers all three, using nothing but arithmetic, probability, and the exact rules recorded in FireWatch's own project files.

1. Where the Data Actually Came From

1.1 Three Public Sources, Pooled Together

FireWatch's training images were not collected from scratch. Three existing, public datasets were combined as the primary source of fire and smoke images: **D-Fire** (Gaia-UFMG, via GitHub), roughly 21,000 labelled fire-and-smoke images and the primary source used - crucially, FireWatch ignores D-Fire's bounding-box annotations entirely and uses only its folder-level labels, since the task here is whole-image classification (does this image contain fire/smoke/neither), not locating exactly where in the image a flame sits. The **Kaggle FIRE Dataset** (roughly 1,000 images) served as a quick baseline while the much larger D-Fire download was still in progress. **BoWFire** (roughly 466 images) contributed additional hard negatives - image scenarios that superficially resemble fire but are not.

1.2 The Three-Class Design, and Why Three, Not Two

The classifier has three output classes, planned at roughly these proportions: neutral (approximately 8,000 images - normal rooms, kitchens, outdoor scenes), smoke (approximately 5,000 - smoke present, no visible flame), and fire (approximately 5,000 - visible flame). The project's own planning notes give the exact reasoning for using three classes rather than a simpler fire-or-not binary: 'smoke-before-flame is the early-warning promise of the product' - a system that only recognizes fire once flame is already visible has given up the single biggest advantage a smoke detector-like early warning could offer.

1.3 Hard Negatives - the Deliberately Hand-Collected Data

Public fire/smoke datasets like D-Fire are built almost entirely from unambiguous fire and non-fire photos - they contain almost no examples of things that merely *look* like fire without being fire. This is exactly why student-built fire detectors tend to false-alarm constantly on ordinary household scenes: sunsets, warm indoor lighting, orange clothing, kitchen steam. FireWatch's plan explicitly calls this out: 'public datasets contain almost no tricky negatives. This is exactly why student fire detectors false-alarm constantly.' The response was a deliberate, targeted data collection effort - six specific hard-negative categories, gathered by automated image search (the ddgs / DuckDuckGo image search package) with every image manually reviewed and validated before use, all labelled neutral.

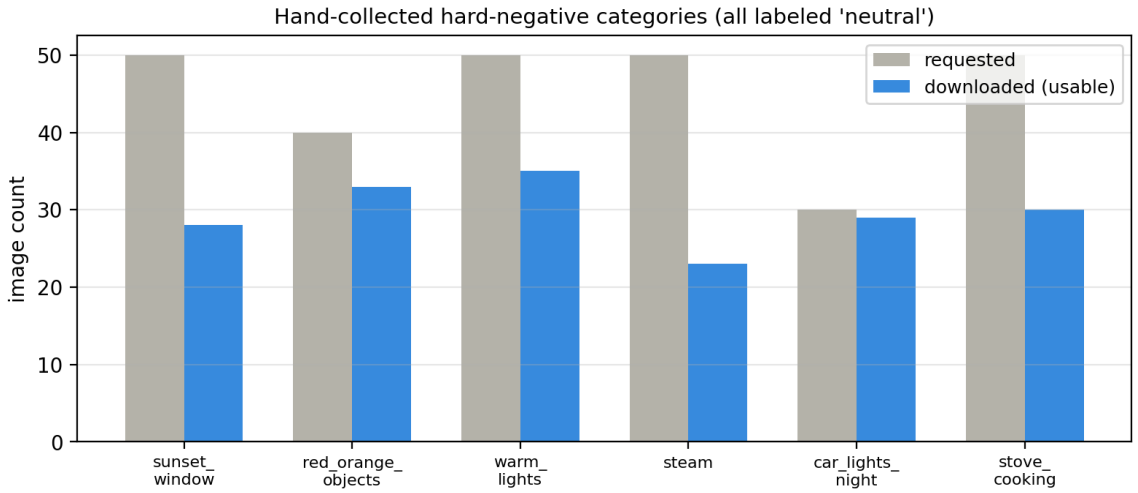


Figure 1.1 - The six hand-collected hard-negative categories and their real, measured download counts. Every one of these images is labelled neutral - the point is teaching the model what fire is NOT, using things that superficially resemble it.

Notice from Figure 1.1 that download counts fall short of requested counts in every category - steam fell furthest short (23 of 50 requested, a 46 percent shortfall). This is a real, disclosed limitation of automated image-search-based collection: a meaningful fraction of search results either fail to download (dead links, hotlink protection returning HTTP 403 errors) or fail basic validation (corrupted files). The project's logs record this honestly rather than treating it as a hidden gap.

1.4 Every Source, Pooled Into One Dataset

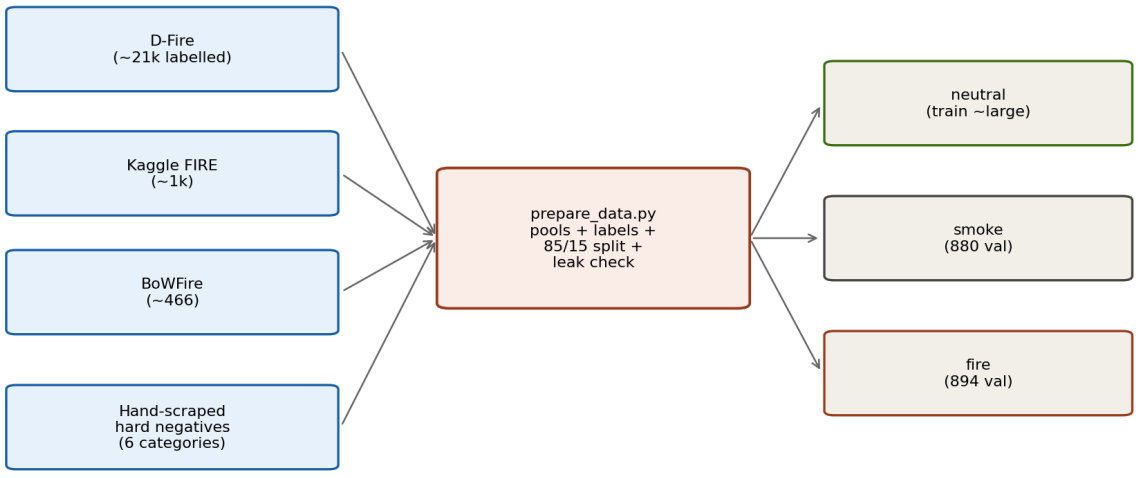


Figure 1.2 - Every data source, from three public fire/smoke datasets to six hand-scraped hard-negative categories, pooled and split into the three training classes by a single script, prepare_data.py.

The final production split (rebuilt and leak-checked by both filename and content hash) contains 18,678 training images and 3,296 validation images (1,522 neutral, 880 smoke, 894 fire) across eight underlying categories once every source and hard-negative batch is counted individually. Section 2 explains exactly how this pooling and splitting happens.

Check your understanding: *given that public datasets like D-Fire contain almost no tricky negatives, and FireWatch's own adversarial testing (documented in the MobileNetV3 bridge document, Section 10) found steam remained a false-positive source even after the v4 retrain - what does the steam category's 46 percent download shortfall in Figure 1.1 suggest about a possible, unconfirmed contributing factor to that persistent weakness?*

2. From Raw Images to Labeled Training Examples

2.1 Deriving a Label From D-Fire's Bounding Boxes

D-Fire ships as an object-detection dataset - each image comes with a YOLO-format label file listing bounding boxes and a class id for anything detected in that image (0 for smoke, 1 for fire). But FireWatch's classifier does not do object detection; it needs one single label per whole image. prepare_data.py derives that single label with a simple, explicit rule: no label file, or an empty one, becomes neutral; a label file containing only class 0 boxes (smoke, no fire) becomes smoke; any label file containing at least one class 1 box (fire) becomes fire - with fire taking priority whenever both smoke and fire boxes appear together in the same image (a reasonable choice, since an image with visible flame is a fire-class image for this task's purposes even if smoke is also present).

2.2 Folding In the Hard Negatives

The six hard-negative categories from Section 1.3 are stored as separate per-category subfolders under a hard_negatives directory, not as a single flat folder. prepare_data.py walks this directory recursively (using rglob, not a simple top-level file listing) and folds every image found, regardless of which of the six categories it came from, into the single neutral class - the category folder names themselves exist only for the project's own record-keeping and per-category count reporting, not as separate model classes.

2.3 The Stratified 85/15 Split

Once every image has a single class label (neutral, smoke, or fire), the full pool is split into a training set and a validation set using an 85/15 **stratified** split - stratified meaning the 85/15 ratio is applied separately within each class, not to the pool as a whole. This matters: if the split were applied to the whole pool at once without stratifying, a class with fewer total images (like fire) could easily end up under- or over-represented in the validation set purely by chance, making the reported validation accuracy less trustworthy. A fixed random seed (42) is used so the exact same split can be reproduced on a future run of the same script.

2.4 The Leakage Check - a Real Bug That Was Caught and Fixed

A subtle but serious problem in any train/validation split is **data leakage** - the same image, or a near-duplicate of it, appearing in both the training set and the validation set. If this happens, validation accuracy becomes an overly optimistic, dishonest estimate of real-world performance, because the model may simply be recognizing an image it already memorized during training rather than genuinely generalizing to new, unseen fire scenes. FireWatch's project report documents exactly this kind of bug being caught: a content-hash leakage check (comparing images by their actual pixel content, not just filename) found 4 stock photos that were byte-identical across the stove_cooking (labelled neutral) and gas_stove_flame (labelled fire) categories in an earlier version of the dataset - the exact same photograph had been independently downloaded once for each category and given two contradictory labels. This was resolved by treating the stove flame version as fire (the correct label, since it shows a flame) and removing the duplicate neutral copies from the training set entirely. The production split is described as leak-free 'by filename AND content hash' - meaning both a

simple filename comparison and a much stronger pixel-content comparison were run, and neither found any remaining leakage.

This leakage check is a direct, practical consequence of a concept the CNN guide covered only in passing: a model's validation accuracy is only meaningful if the validation set genuinely represents unseen data. A single duplicated image sitting in both splits does not just add noise - it can actively inflate a reported accuracy number in a way that would not hold up once the model meets a truly new, real-world camera frame.

Check your understanding: *if the stove-flame/stove-cooking duplicate had been resolved the opposite way - keeping both copies, one as neutral and one as fire, both in the training set - what specific problem would this create for the model during training, thinking back to what a loss function (CNN guide, Section 10) is actually trying to do with a single image's label?*

3. Data Augmentation - Manufacturing Variety From Fixed Photos

3.1 The Intuition

18,678 training images is a fixed, finite number - once a model has seen every one of them several times over several epochs (CNN guide, Section 13.2), it risks starting to memorize their exact pixel arrangements rather than learning the general, transferable concept of 'what fire looks like.' **Data augmentation** fights this by applying small, random, label-preserving transformations to each training image every time it is loaded - the same photo of a flame might be shown to the model cropped slightly differently, flipped left-right, or with its colors nudged, on different epochs. The underlying fire is still fire; only its exact pixel presentation changes. This manufactures a much larger effective variety of training examples from the same fixed pool of source photos.

3.2 The Specific Augmentations FireWatch Uses

FireWatch's training pipeline applies four specific augmentations to every training image, chosen to simulate realistic variation a real camera might actually encounter: a random-resized crop (scale 0.6 to 1.0 of the original image), a horizontal flip, and mild color jitter (brightness/contrast/saturation adjusted by up to plus-or-minus 0.2, but hue adjusted by only plus-or-minus 0.02 - a much tighter range). Validation images receive none of these - only a plain 224x224 resize and the same normalization used at inference time (Section 4), with no randomness at all.

3.3 The Math - What 'Scale 0.6 to 1.0' Actually Means

A random-resized crop first picks a random rectangular region of the original image whose area is somewhere between 60 percent and 100 percent of the original image's total area, then resizes that cropped region up to the standard 224x224 input size. Worked example: suppose an original training photo is 800x600 pixels, giving a total area of 800 × 600 = 480,000 square pixels. If the random scale factor drawn for this particular load happens to be 0.7 (within the allowed 0.6-1.0 range), the crop region's target area is 0.7 × 480,000 = **336,000 square pixels** - roughly a 650x517 rectangle sitting somewhere inside the original 800x600 photo (the exact aspect ratio and position are also randomized within reasonable bounds). This simulates a fire appearing nearer, farther, or off-center compared to how the original photo framed it - variety a real deployed camera would genuinely encounter, since a fire will not always appear perfectly centered and full-frame the way a curated dataset photo might.

3.4 Why Hue Jitter Is Kept Ten Times Tighter Than Everything Else

Brightness, contrast, and saturation are jittered by up to ±0.2 (a 20 percent range), but hue is jittered by only ±0.02 (a 2 percent range) - ten times tighter. This is a deliberate, task-specific choice, not an oversight. The project's report states the reasoning directly: hue is 'exactly what separates fire from steam and sunsets in this task; aggressive hue jitter would destroy the signal the model needs to learn.' Hue is the property of color that distinguishes orange/red (fire's characteristic color) from white/gray (steam's characteristic color) from the

more golden tones of a sunset - if training randomly shifted a fire image's hue by a large amount, the augmented version might visually drift toward looking like steam or a sunset while still being labelled fire, actively teaching the model an incorrect association between color and label. Brightness, contrast, and saturation can vary much more freely because they do not carry this same specific, label-relevant information for this particular task.

This is a clean, concrete example of a general principle worth remembering whenever choosing augmentation settings for any project: an augmentation is only safe if it does not scramble the specific visual signal the model actually needs to tell its classes apart. What counts as a safe amount of jitter is not a universal constant - it depends entirely on which visual property carries the label-relevant information for the task at hand.

3.5 Why Validation Images Get None of This

Validation images are deliberately given only a plain resize and the same normalization used at real inference time - no cropping, no flipping, no color jitter. The report is explicit that this choice is 'deliberately a direct resize rather than resize-then-center-crop' specifically because it is 'chosen to exactly mirror what edge/vision.py does to a live webcam frame at inference time, so validation accuracy is an honest estimate of deployment accuracy rather than a jittered training-time metric.' If validation images used augmentation, a reported validation accuracy number would be measuring how well the model performs on artificially varied images that no real deployment would ever actually produce, rather than how it will perform on real camera frames.

Check your understanding: *if FireWatch had instead used a wide hue jitter range (say ±0.3) during training, and the model consequently learned to associate a broad range of hues - including some closer to steam's whitish-gray - with the label fire, what specific kind of real-world false alarm would this most plausibly make worse?*

4. What Happens to a Live Camera Frame Before It Reaches the Model

4.1 Why This Has to Match Training Exactly

The MobileNetV3 bridge document's Section 9.4 already explained why: every filter inside the network learned its weights relative to a specific numeric range of input values. If a live camera frame is prepared any differently than the images the model was trained and validated on, its pixel values will sit outside the distribution every filter expects, and predictions become unreliable in ways that are difficult to predict or debug. edge/vision.py's preprocessing function, _preprocess(), exists specifically to guarantee this match, step for step.

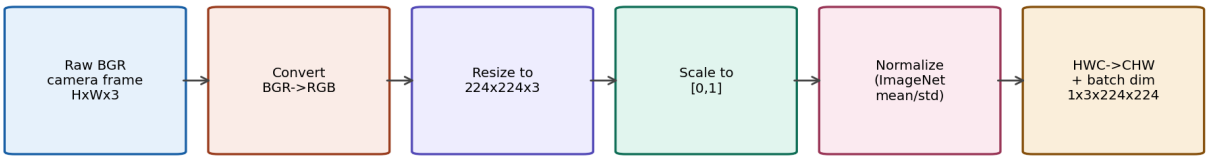


Figure 4.1 - The exact preprocessing pipeline applied to every live camera frame before it reaches the model, matching the validation-time transform used during training exactly.

4.2 Step by Step, With Real Numbers

Step 1 - BGR to RGB. OpenCV, the library used to read frames from the webcam, stores color images in Blue-Green-Red channel order by convention - the opposite of the more common Red-Green-Blue order most image-processing and deep learning tools (including the PyTorch/torchvision pipeline this model was trained with) expect. cv2.cvtColor swaps the channel order. Worked example: a pure, fully saturated red pixel arrives from the camera as BGR = (0, 0, 255) (zero blue, zero green, maximum red); after this conversion it becomes RGB = (255, 0, 0). Skipping this step would not crash anything - it would silently feed the model color-channel-swapped input, which (based on how the model's very first convolution filters, MobileNetV3 bridge document Section 7.2's first row, were trained) would corrupt exactly the color signal Section 3.4 just established as label-critical for telling fire apart from steam and sunsets.

Step 2 - Resize to 224x224. Whatever resolution the webcam captures at, the frame is resized to exactly 224x224 - the input size traced through the entire network in the MobileNetV3 bridge document's Section 8. This value is read from config.yaml's input_size setting rather than being hardcoded, so it is one clearly-labeled tunable rather than a magic number buried in code.

Step 3 - Scale to [0, 1]. Raw pixel values from a camera or an image file are integers from 0 to 255 (an 8-bit color channel). Every value is divided by 255 to bring it into the 0-to-1 range expected by the

normalization step that follows. Worked example: a raw red channel value of 200 becomes 200/255 ≈ **0.784**.

Step 4 - Normalize with ImageNet statistics. Each of the three channels is independently shifted and scaled using the fixed ImageNet mean and standard deviation values referenced throughout the MobileNetV3 bridge document (mean = [0.485, 0.456, 0.406], std = [0.229, 0.224, 0.225] for red, green, blue respectively). The formula for each channel is:

normalized_value = (scaled_value - mean) / std

Worked example, continuing the red channel value from Step 3: normalized_red = (0.784 - 0.485) / 0.229 = 0.299 / 0.229 ≈ **1.306**. Notice the result is no longer confined to 0-to-1 - normalization can and does produce negative values (for a pixel darker than the ImageNet average) or values above 1 (for a pixel brighter than average, as in this example). This is expected and correct - the whole point of this step is to re-center and re-scale pixel values onto the exact numeric distribution the pretrained filters were originally tuned against, not to keep them in any particular bounded range.

Step 5 - HWC to CHW, plus a batch dimension. OpenCV and most image libraries store an image with shape Height x Width x Channels (HWC) - for a 224x224 RGB image, that is 224x224x3. PyTorch (and by extension the exported ONNX model) expects Channels x Height x Width (CHW) instead - 3x224x224 - with the channel dimension moved to the front. Finally, a batch dimension of size 1 is added at the very front, giving a final tensor shape of **1x3x224x224** - even though only one single frame is being processed, the model's input signature always expects a batch dimension, because the exact same network can process many images at once during training by simply making that first dimension larger than 1.

This five-step pipeline is applied to every single live camera frame, roughly 30 times per second at FireWatch's measured frame rate, and is required to be byte-for-byte consistent with the Resize((224,224)) -> ToTensor() -> Normalize() transform used on validation images during training - any divergence between the two would mean validation accuracy no longer honestly predicts real deployment accuracy.

Check your understanding: *if Step 1 (the BGR-to-RGB conversion) were accidentally skipped in production, would a genuinely orange/red fire in the frame most likely look, to the model, more like a very different-hued object - and if so, roughly what hue would a swapped-channel version of orange shift toward?*

5. From Raw Logits to a Decision - Softmax, Thresholds, and the Argmax-AND Rule

5.1 The Model's Raw Output Is Not the Final Answer

Following the preprocessed 1x3x224x224 tensor from Section 4 through the full MobileNetV3-Small forward pass (MobileNetV3 bridge document, Section 8) produces 3 raw numbers - logits - one per class. The class-to-index mapping is fixed by the trained model file itself, hardcoded in vision.py rather than left in config.yaml, specifically because 'it's a fixed property of the trained model file (changing it requires retraining, not a config edit).' The confirmed mapping is:

Index	Class
0	fire
1	neutral
2	smoke

This ordering matters enormously and is a genuine, easy-to-make mistake: index 0 is fire, not neutral. Any code (or person) that assumed a more intuitive-seeming order (e.g. neutral first, since it might feel like the 'default' class) would silently read the wrong probability for the wrong class. FireWatch's own logs record this exact mapping was confirmed directly from train/export_onnx.py's printed checkpoint['class_to_idx'] dictionary, rather than assumed.

5.2 Softmax - Turning 3 Logits Into 3 Probabilities

Exactly as covered in the CNN guide's Section 5.6, softmax converts the 3 raw logits into 3 probabilities that sum to 1. Worked example: suppose one live frame produces raw logits [fire=1.2, neutral=-0.5, smoke=0.3] (in index order 0, 1, 2). Exponentiating: $e^{1.2} \approx 3.320$, $e^{-0.5} \approx 0.607$, $e^{0.3} \approx 1.350$. Summing: 3.320 + 0.607 + 1.350 = 5.277. Dividing each by the sum: P(fire) = 3.320/5.277 ≈ **0.629**, P(neutral) = 0.607/5.277 ≈ **0.115**, P(smoke) = 1.350/5.277 ≈ **0.256**. These three numbers sum to 1.000, and this is exactly the dictionary shape vision.py's predict() function returns: {'fire': bool, 'smoke': bool, 'p_fire': float, 'p_smoke': float}.

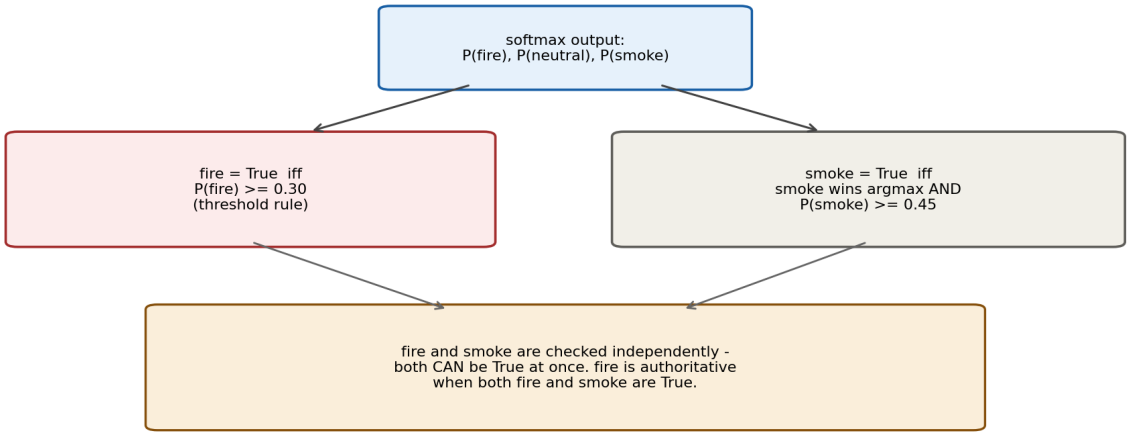


Figure 5.1 - fire and smoke are decided by two different, independently-applied rules on top of the same softmax output - not by simply picking whichever class has the single highest probability.

5.3 The Fire Rule - a Simple Threshold

fire is declared True if and only if P(fire) is at least 0.30 - notably not 0.50, and not simply 'fire is the argmax winner.' This threshold was chosen via an explicit threshold sweep (trying many candidate thresholds against the validation set and measuring recall and precision at each one), picking 'the lowest threshold that still clears both the project's fire-recall and fire-precision bars.' Using our worked example above, P(fire) ≈ 0.629, which clears 0.30 easily, so fire = **True** for this frame - even though, notice, fire is also the argmax winner here (0.629 is the largest of the three probabilities), so this particular example does not yet show why a threshold below 0.50 matters. It matters specifically in borderline cases: a frame where P(fire) = 0.35, P(neutral) = 0.40, P(smoke) = 0.25 would have fire lose the argmax vote to neutral, yet still clear the 0.30 threshold and be declared fire = True. The threshold rule deliberately catches real fires the model is only moderately, not overwhelmingly, confident about - accepting a real cost (some extra false alarms) in exchange for a benefit the project judged more important for a safety-critical system: not missing a real fire just because the model's confidence fell short of an arbitrary 50 percent bar.

5.4 The Smoke Rule - Argmax AND a Threshold, Together

smoke uses a different, stricter rule than fire: smoke is declared True only if smoke both wins the 3-way argmax vote (has the single highest probability of the three) **and** clears its own threshold of 0.45. This two-part rule - referred to in the project's own logs as the 'argmax-AND' rule - was not the original design. The logs document its exact origin: smoke originally used plain argmax alone, with no threshold at all, and 'live testing showed argmax alone let smoke spuriously win 3-way splits at 0.28-0.44' - meaning a frame could be called smoke even when the model's actual confidence in that verdict was as low as 28 percent, simply because the other two classes happened to be even lower. Adding the 0.45 threshold on top of argmax removes exactly this failure mode by construction: even if smoke technically wins the 3-way vote, a frame is not declared smoke unless the model's confidence in smoke specifically also clears 0.45.

Worked example: suppose a frame produces P(fire) = 0.22, P(neutral) = 0.38, P(smoke) = 0.40. Smoke wins the 3-way argmax vote (0.40 is the largest of the three), but 0.40 does not clear the 0.45 threshold,

so smoke = **False** for this frame under the argmax-AND rule - even though it would have been declared smoke = True under the old, argmax-only rule. This exact kind of frame is precisely what the rule change was designed to catch.

The project's logs also record that this threshold was a deliberate developer choice made over a data-driven recommendation: a threshold sweep tool (eval/smoke_threshold_sweep.py) suggested 0.50 as a candidate value, but the developer chose 0.45 instead, and this choice, along with its reasoning, is recorded in config.yaml and the project logs rather than silently overriding the sweep's suggestion without a trace.

5.5 Both Can Be True at Once - and Fire Wins

Because fire and smoke are each checked by their own independent rule against the same three probabilities, it is entirely possible for both fire = True and smoke = True to come back for the same frame - for instance, if P(fire) is 0.32 (clearing fire's 0.30 threshold) while smoke still happens to win the argmax vote and separately clears 0.45. FireWatch's logs flag this explicitly as a consequence worth understanding: 'callers must treat fire as authoritative' whenever both come back true at once. This is not a bug or an oversight - it is a deliberate consequence of evaluating two independent safety-relevant rules against the same underlying probabilities, with an explicit, documented tie-breaking priority (fire over smoke) for whatever downstream logic (the fusion and alerting pipeline) consumes this result.

Check your understanding: *using the fire rule (threshold only, no argmax requirement) and the worked probabilities P(fire)=0.35, P(neutral)=0.40, P(smoke)=0.25 from Section 5.3, would smoke ever be declared True for this same frame under the argmax-AND rule, and why or why not?*

6. Temporal Voting - Turning One Noisy Frame Into a Stable Verdict

6.1 The Intuition

Section 5's rules operate on a single frame in isolation, and a single frame can be noisy - a camera autofocus adjustment, a momentary lighting flicker, or an unusual reflection can push one frame's P(fire) above 0.30 even with no real fire present. Rather than sounding an alarm the instant any one frame clears its threshold, FireWatch requires a pattern of agreement across several consecutive frames before trusting a verdict - the same intuition the CNN guide's own opening sections used when explaining why nearby pixels matter to each other: a real fire persists and stays consistent across many consecutive video frames the way a single-frame camera glitch does not.

6.2 The Exact Rule, and Where the Numbers Actually Came From

The project's own planning document states this rule in a single, precise line of pseudo-math: alarm holds true if and only if the count of the last M frames where p_fire exceeds tau is at least N. FireWatch's TemporalVoter implements exactly this: a sliding window of the M = 8 most recent frames, a per-frame vote cast whenever that frame's P(fire) clears a confidence gate of tau = 0.70 (a separate, stricter threshold from Section 5.3's 0.30 decision threshold, since this gate feeds a smoothing mechanism rather than being the final decision itself), and a smoothed, trusted signal of True only once at least N = 5 of the 8 frames in the current window have voted positive.

It is worth being precise about where M=8, N=5, and tau=0.70 actually came from, since this is a detail easy to get wrong: the project's own plan explicitly labels these as **start values**, not numbers derived from any formal sweep or optimization process before the system was ever run. They were later stress-tested, not simply assumed forever: when the TV/laptop-fire adversarial scenario (MobileNetV3 bridge document, Section 10) proved a persistent, bounded false-positive source, the project explicitly tried votes_needed (N) as a remediation lever, sweeping candidate values from 5 through 8 with a dedicated diagnostic script. The documented outcome: this sweep was 'exhausted as a remediation lever' - meaning no value of N in that range brought the TV/laptop-fire false-alarm count down to the project's required bar, so N was left at its original value of 5, and the actual fix for that specific false-positive source was implemented one stage further downstream, in the fusion logic (capping any vision-only evidence at a WARNING level, never allowing it alone to reach a CRITICAL alert) rather than in the temporal voter itself.

This is a useful, honest example of how a starting value in an engineering system does not automatically become the 'right' value just because it works well in most cases - it gets kept only after a real attempt to find something better failed to justify a change, and the actual remaining weakness gets acknowledged and handled elsewhere, rather than the original number being silently presented as if it had always been carefully optimized.

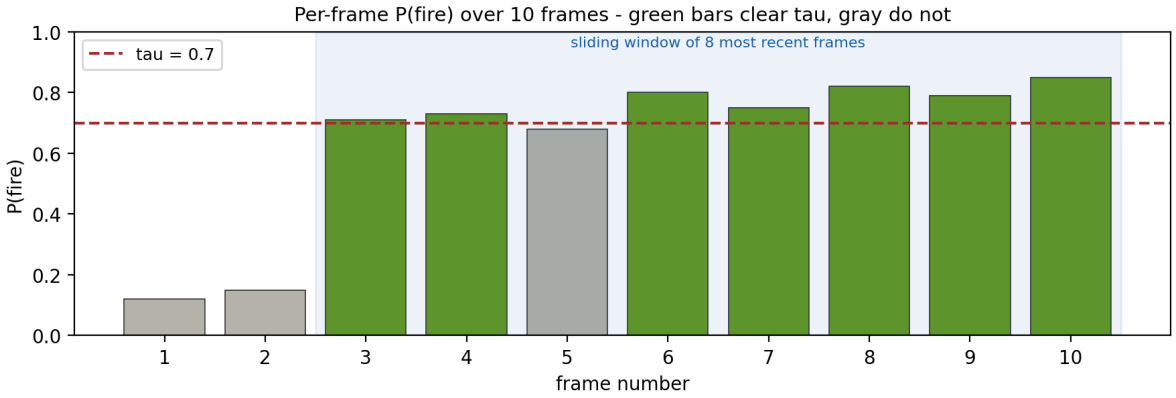


Figure 6.1 - Ten consecutive frames' P(fire) values. Green bars clear tau=0.70; gray bars do not. The shaded region marks the sliding window of the 8 most recent frames (frames 3 through 10).

Worked example, using Figure 6.1's values. Looking at the 8-frame window (frames 3 through 10): frame 3 (0.71, clears tau), frame 4 (0.73, clears), frame 5 (0.68, does NOT clear - just under tau), frame 6 (0.80, clears), frame 7 (0.75, clears), frame 8 (0.82, clears), frame 9 (0.79, clears), frame 10 (0.85, clears). Counting positive votes: frames 3, 4, 6, 7, 8, 9, 10 all clear tau - that is **7 positive votes** out of the 8-frame window. Since 7 is at least the required N = 5, the temporal voter's smoothed output is **True** at this point, even though frame 5 alone did not individually clear the per-frame gate - a single dip below tau does not undo an otherwise consistent pattern.

This is precisely the behavior confirmed in FireWatch's own live testing logs: 'transient match-flame spikes (peak 0.32) never crossed tau, votes stayed 0/8' for a brief, non-sustained flame, while 'a sustained match flame climbed to ALARM: True in 5 frames (p_fire 0.73 to 0.93, votes 1/8 to 5/8)' for a real, continuing flame - and critically, the measured timing for that transition was 'approximately 0.17 seconds at measured FPS,' meaning the 5-of-8 rule does not meaningfully delay a genuine, sustained fire signal, precisely because a real fire keeps producing high P(fire) values on essentially every subsequent frame once it starts.

The same voting logic and window size are applied independently to smoke's decision (Section 5.4's rule feeds its own separate vote count), so a stable, non-transient smoke signal is required separately from a stable fire signal, each smoothing out single-frame noise on its own class.

Check your understanding: *if only 4 of the 8 frames in a window cleared tau (one fewer than the required N=5), and the very next new frame arriving also clears tau (pushing one of the older, non-clearing frames out of the 8-frame window), would the smoothed output become True on that next frame - and what does this tell you about how quickly a borderline pattern can flip the voter's output as the window slides forward one frame at a time?*

7. The Complete End-to-End Trace, One Frame at a Time

7.1 Every Step, in One Place

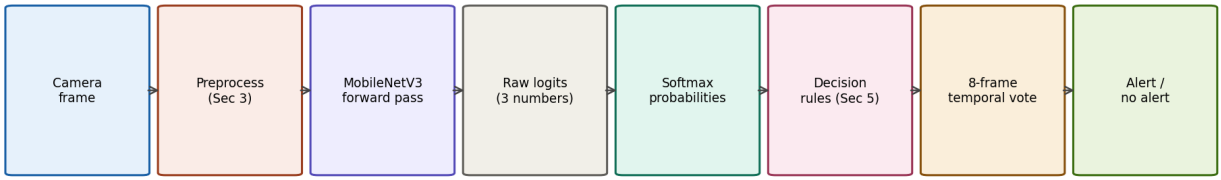


Figure 7.1 - The complete journey of a single camera frame, from raw pixels to a final alert decision, combining every mechanism covered in this document and the two companion documents.

Let's walk through one complete, concrete pass, tying together every section of this document.

1. Camera captures a frame. A raw BGR image arrives from edge/camera.py, at whatever resolution the webcam provides.

2. Preprocessing (Section 4). BGR to RGB, resize to 224x224, scale to [0,1], normalize with ImageNet statistics, reorder to CHW, add a batch dimension - final shape 1x3x224x224.

3. Forward pass through MobileNetV3-Small (MobileNetV3 bridge document, Section 8) - the tensor passes through the initial convolution, all 11 bottleneck blocks, the channel expansions, global average pooling, and the fine-tuned Linear(1024 to 3) head, producing 3 raw logits.

4. Softmax (Section 5.2) converts the 3 logits into 3 probabilities summing to 1, correctly indexed as fire=0, neutral=1, smoke=2.

5. Per-frame decision rules (Sections 5.3-5.4) apply independently: fire = True iff P(fire) >= 0.30; smoke = True iff smoke wins argmax AND P(smoke) >= 0.45.

6. Temporal voting (Section 6) adds this frame's result into an 8-frame sliding window for each class separately, and only reports a smoothed, trusted signal once at least 5 of the 8 most recent frames agree.

7. Downstream fusion and alerting (outside the scope of this document, but the next stage in FireWatch's actual pipeline) combines this smoothed vision signal with the gas and temperature sensor readings before deciding whether to sound a physical alarm or escalate to the response agent.

Every single number that appears anywhere in this seven-step chain - the 224x224 input size, the ImageNet mean and std values, the 0.30 fire threshold, the 0.45 smoke threshold, the 5-of-8 temporal vote, the tau=0.70

per-frame gate - is a real, documented, tuned value from FireWatch's own project files, not an illustrative placeholder. Changing any one of them changes the system's real, measured behavior, and each one was arrived at through a specific investigation documented in the project's own logs.

8. Why These Specific Rules Exist - the False-Positive Investigations Behind Them

8.1 The Bright, Textured Wall That Broke Smoke Detection

The single clearest before/after result in FireWatch's entire project traces directly back to the data and rules covered in this document. Live testing found that a bright, textured wall under strong lighting produced a sustained smoke false-positive - the temporal voter's smoke vote count saturated at 8-of-8 (Section 6's mechanism working exactly as designed, but locking onto a wrong signal, since the false trigger was itself sustained and consistent across frames, not a brief transient). Because the false signal was sustained rather than transient, the 5-of-8 temporal voting rule from Section 6 could not filter it out by design - voting only protects against brief, inconsistent noise, not against a genuinely sustained but wrong per-frame signal. The real fix had to happen further back in the pipeline, in the training data itself.

The project's response was exactly the hard-negative-collection process described in Section 1.3 of this document, applied a second time: 30 new frames of the specific bright_light_textured_wall scenario were collected and added to the neutral training class, and the model was retrained (this is the v3-to-v4 retrain referenced throughout the MobileNetV3 bridge document). The measured result: the held-out validation clip's smoke-frame rate dropped from 98.0 percent to 0.0 percent. This is a direct demonstration of a idea from Section 1 of this document taken to its logical conclusion - a model can only learn to correctly reject a specific kind of false positive if the training data actually contains labelled examples of that specific scenario. No amount of downstream rule-tuning (thresholds, argmax-AND logic, or temporal voting) can substitute for the training data itself containing the relevant hard negative.

8.2 Why Fire and Smoke Use Different Threshold Philosophies

Section 5 showed that fire uses a plain threshold (P(fire) >= 0.30, no argmax requirement) while smoke uses a stricter argmax-AND rule. This asymmetry is not an oversight - it reflects two different histories. Fire's threshold was swept and tuned from early in the project (Phase 2), specifically to widen the model's fire-recall margin over a required 0.95 floor - a deliberately permissive threshold for the class the project considers most safety-critical to never miss. Smoke's rule, by contrast, started as plain argmax with no threshold at all, and the argmax-AND rule was added later, specifically in reaction to the 0.28-0.44 false-positive band documented in Section 5.4 - smoke needed a stricter, not a more permissive, rule once live testing revealed low-confidence spurious wins were a real, observed problem for that class specifically.

8.3 The Honest Limitations Still on Record

FireWatch's own report keeps an explicit, disclosed limitations list rather than presenting the system as fully solved. Two directly concern the data and rules in this document: steam remains a known, if much-improved, smoke false-positive source (21 sustained smoke-WATCH events on v3, reduced to 5 on v4, but not eliminated) - notice this connects directly back to Section 1.3's observation that the steam hard-negative category fell furthest short of its download target of any category. A television or laptop screen showing fire footage also reliably triggers a vision-side warning (6 alarms on the standard adversarial test clip) - bounded by the downstream fusion logic (requiring gas sensor corroboration before ever reaching a critical alert) but

not eliminated at the vision-model level, since the model has no way to distinguish a real flame from a photographically accurate video of one using only the visual patterns this document's pipeline gives it access to.

9. What Recall and Precision Actually Mean - the Numbers Behind Every Metric in This Project

Every accuracy figure quoted throughout this document and its two companions - 0.9575 fire recall, 0.8664 fire precision, 0.8511 smoke recall - is built from just four counts, and it is worth defining them precisely once rather than treating them as unexplained percentages.

Take fire as the example class. For every validation image, compare what the model predicted against the true label. A **true positive (TP)** is a real fire image the model correctly called fire. A **false negative (FN)** is a real fire image the model missed - it called something else instead. A **false positive (FP)** is a non-fire image the model incorrectly called fire. A **true negative (TN)** is a non-fire image the model correctly did not call fire.

$$recall = TP / (TP + FN)$$

Recall answers: *of every real fire that existed in the validation set, what fraction did the model actually catch?* A recall of 1.0 would mean the model missed zero real fires (regardless of how many false alarms it also raised); a recall of 0.50 would mean it missed half of them. This is precisely why FireWatch's project treats fire recall as its single most safety-critical number - a missed real fire is a far worse outcome for this system than a false alarm.

$$precision = TP / (TP + FP)$$

Precision answers a different question: *of every image the model called fire, what fraction were actually fire?* A precision of 1.0 would mean the model never cried wolf - every single fire alert it ever raised was correct; a lower precision means a real user would experience more false alarms for every genuine one.

Worked example, using production checkpoint v4's real, reported numbers on the 894 true-fire images in the validation set: fire recall at the 0.30 threshold is 0.9575. Multiplying back out: TP + FN = 894 (every true fire image), so TP = 0.9575 × 894 ≈ **856**, meaning FN = 894 - 856 = **38** real fire images the production model still misses out of every 894 in the validation set. Fire precision at the same threshold is 0.8664 - using precision = TP / (TP + FP), and TP ≈ 856: 0.8664 = 856 / (856 + FP), so (856 + FP) = 856 / 0.8664 ≈ 988, giving FP ≈ **132** - roughly 132 non-fire validation images the production model incorrectly flags as fire at this threshold.

This is exactly the concrete, human-readable meaning behind Section 5.3's fire threshold discussion: choosing 0.30 rather than 0.50 is a choice that trades a specific, countable number of extra false positives (contributing to that ~132 figure) for a specific, countable reduction in missed real fires (keeping that ~38 figure as low as it is). Neither number is ever zero - the threshold sweep referenced in the MobileNetV3 bridge document's Section 10 exists precisely to find a threshold where both counts are judged acceptable together, not to eliminate either one entirely.

Macro F1 (also quoted in the project's report, 0.9065 for v4) combines recall and precision into one single number: F1 = 2 × (precision × recall) / (precision + recall), computed per class and then averaged

across all three classes (hence 'macro'). It is a useful single-number summary for comparing checkpoints at a glance (as the v3-versus-v4 comparison table in the MobileNetV3 bridge document does), but it is not what any of this project's actual decision thresholds were tuned against - those were tuned against recall and precision directly, at specific, separately-chosen thresholds per class, exactly as Section 5 of this document describes.

Check your understanding: *if a future retrain increased fire recall to 0.99 but dropped fire precision to 0.60, roughly how many of the 894 true fire images would still be missed, and roughly how many false alarms would you expect out of every 100 fire alerts the model raises - and would you consider that an improvement for a safety system, a regression, or a genuine trade-off depending on priorities?*

10. Quick Reference - Every Number, in One Place

10.1 Dataset Numbers

Quantity	Value
Training images	18,678
Validation images	3,296 (neutral 1,522 / smoke 880 / fire 894)
Train/val split ratio	85/15, stratified by class, seed 42
Primary source	D-Fire (~21k images), folder labels only, boxes ignored
Secondary sources	Kaggle FIRE (~1k), BoWFire (~466)
Hard-negative categories	sunset_window, red_orange_objects, warm_lights, steam, car_lights_night, stove_cooking
Augmentation (train only)	random-resized crop 0.6-1.0, h-flip, brightness/contrast/saturation +/-0.2, hue +/-0.02

10.2 Preprocessing and Decision Numbers

Quantity	Value
Input size	224x224 RGB
Normalization mean (R,G,B)	0.485, 0.456, 0.406
Normalization std (R,G,B)	0.229, 0.224, 0.225
Class index mapping	0=fire, 1=neutral, 2=smoke
Fire decision rule	fire = True iff P(fire) >= 0.30
Smoke decision rule	smoke = True iff smoke wins argmax AND P(smoke) >= 0.45
Temporal voter window (M)	8 frames
Temporal voter votes needed (N)	5 of 8
Temporal voter per-frame gate (tau)	0.70
Both fire and smoke True	possible; fire is authoritative

10.3 Vocabulary Glossary

Data leakage - The same or near-duplicate image appearing in both the training and validation sets, inflating reported accuracy dishonestly.

Stratified split - Splitting a dataset so a fixed ratio (e.g. 85/15) is applied separately within each class, not to the whole pool at once.

Hard negative - A training example that superficially resembles the positive class but is correctly labelled negative, specifically chosen to teach a model what NOT to trigger on.

Data augmentation - Applying small, random, label-preserving transformations to training images to manufacture effective variety from a fixed dataset.

Logit - A model's raw, un-normalized output number for one class, before softmax converts it into a probability.

Argmax - The operation of picking the single class with the highest value among a set of numbers.

Argmax-AND rule - A decision rule requiring a class to both win the argmax vote and separately clear its own confidence threshold.

Temporal voting / smoothing - Requiring a pattern of agreement across several consecutive frames before trusting a single-frame model output.

Sliding window - A fixed-size window (e.g. the 8 most recent frames) that moves forward one frame at a time, always covering only the most recent data.

Threshold sweep - Systematically trying many candidate threshold values against a validation set to find one that best satisfies given accuracy requirements.

End of guide. Together with the CNN fundamentals document and the MobileNetV3 bridge document, this completes the full path from raw pixels and raw training photographs all the way to a real alert decision - every mechanism in between now traced, worked by hand at least once, and grounded in FireWatch's own recorded engineering decisions.